

fitbit2garmin

Bidirectional converter between Fitbit and Garmin data formats.

After years using Fitbit for activity tracking, heart rate monitoring, sleep analysis, and weight logging, migrating to Garmin means leaving your history behind — unless you bring it with you. This project provides tools to translate your health data across platforms.

Tools

[fitbit2garmin](#) →

Convert your Fitbit Google Takeout export into TCX and CSV files that Garmin Connect can import. Supports activities (with heart rate), body composition, sleep records, and daily summaries.

[garmin2fitbit](#) →

Convert Garmin data exports (TCX activities, CSV body/sleep/summaries) into Fitbit's native JSON format — the same structure used by Google Takeout. Useful for archiving or feeding into third-party tools that use the Fitbit Web API.

Sample Data

The `sample/` directory contains representative data for both converters:

```
sample/
├── fitbit2garmin/
│   ├── input/           ← Fitbit export (Google Takeout JSON)
│   └── output/          ← fitbit2garmin output
└── garmin2fitbit/
    ├── input/           ← Garmin export (TCX + CSV)
    └── output/          ← garmin2fitbit output
```

Regenerate all sample output:

```
python -m fitbit2garmin sample/fitbit2garmin/input -o sample/fitbit2garmin/output
python -m garmin2fitbit sample/garmin2fitbit/input -o sample/garmin2fitbit/output
```

Project Structure



Third-Party Sync Tools

If you want all your data (Fitbit history + Garmin daily data) in one place, these tools bridge the gap:

Tool	Platform	Price	What it syncs to Fitbit	What it syncs to Garmin
Health Sync	Android, iOS	~\$4–6 lifetime (one-time)	Steps, HR, sleep, weight, activities via Google Health APIs	Steps, HR, sleep, weight, activities
RunGap	iOS	\$4.99/quarter or \$14.99/year (Swag Bag)	Activities (workouts) to Fitbit	Activities to Garmin Connect
myFitnessSync	iOS	\$9.99/month or \$59.99/lifetime	Steps, weight, body fat, sleep, water (5 fields)	—

Tool	Platform	Price	What it syncs to Fitbit	What it syncs to Garmin
Strava	Web, iOS, Android	Free tier (Summit \$5/mo optional)	—	Auto-sync from Garmin; manual upload from Fitbit

Recommendation: On iOS, **RunGap** + Apple Health gives you a single source of truth. On Android, **Health Sync** with Google Health (Fitbit) serves the same role.

Requires Python 3.10+. No external dependencies.

fitbit2garmin — Converter Documentation

fitbit2garmin

Convert your Fitbit data export to Garmin-compatible formats for import into Garmin Connect.

After years using Fitbit for activity tracking, heart rate monitoring, sleep analysis, and weight logging, migrating to Garmin means leaving your history behind — unless you bring it with you. This tool translates your Fitbit Google Takeout export into files that Garmin Connect can actually import.

Table of Contents

- [Exporting Your Fitbit Data](#)
 - [Fitbit Input Formats](#)
 - [Garmin Output Formats](#)
 - [Feature Comparison: Fitbit vs Garmin](#)
 - [Conversion Mapping](#)
 - [How the Integration Works](#)
 - [Installation](#)
 - [Usage](#)
 - [Importing into Garmin Connect](#)
 - [File Format Specifications](#)
 - [Limitations](#)
 - [FAQ](#)
-

Exporting Your Fitbit Data

Since Google's acquisition of Fitbit, all data exports go through **Google Takeout**.

Step-by-step

1. Go to [Google Takeout](#)
2. Click **Deselect all**, then search for and select **Fitbit**
3. Choose **JSON** format (recommended — CSV also works for some types)

4. Set delivery to email, frequency to single export, and file size to 2 GB
5. Click **Create export**
6. Wait for the email (minutes to hours depending on data volume)
7. Download and extract the ZIP archive

Directory structure

The extracted archive follows this layout:

```
Takeout/  
├─ Fitbit/  
│   ├─ Global Export Data/           ← daily summaries + sensors  
│   │   ├─ heart_rate-2024-01-15.json  
│   │   ├─ sleep-2024-01-15.json  
│   │   ├─ activities-2024-01-15.json  
│   │   ├─ steps-2024-01-15.json  
│   │   ├─ calories-2024-01-15.json  
│   │   ├─ distance-2024-01-15.json  
│   │   └─ weight-2024-01-15.json  
│   ├─ Physical Activity/           ← detailed workout records  
│   │   └─ activities-2024-01-15.json  
│   ├─ Body/                       ← weight & body composition  
│   │   └─ weight-2024-01-15.json  
│   └─ Personal & Account/         ← profile, devices, settings  
└─ ...
```

Point the converter at the `Takeout/Fitbit/` or any directory containing these JSON files.

Fitbit Input Formats

The converter reads all common Fitbit JSON structures found in Google Takeout. Below are the actual formats with real sample data.

Heart Rate (`heart_rate-*.json`)

Two formats are supported:

Intraday samples (most common in Takeout):

```
[  
  {"dateTime": "01/15/24 07:30:00", "value": {"bpm": 95, "confidence": 3}},  
  {"dateTime": "01/15/24 07:35:00", "value": {"bpm": 112, "confidence": 3}}  
]
```

Daily summary with zones (older format):

```

{
  "activities-heart": [
    {
      "dateTime": "2024-01-15",
      "value": {
        "restingHeartRate": 62,
        "heartRateZones": [
          {"name": "Out of Range", "min": 30, "max": 90, "minutes": 180},
          {"name": "Fat Burn", "min": 91, "max": 140, "minutes": 60}
        ]
      }
    }
  ]
}

```

Date formats accepted: MM/dd/yy HH:mm:ss , YYYY-MM-dd HH:mm:ss , YYYY-MM-ddTHH:mm:ss , MM/dd/YYYY HH:mm:ss .

Sleep (sleep-*.json)

```

{
  "sleep": [
    {
      "dateOfSleep": "2024-01-15",
      "duration": 29400000,
      "efficiency": 93,
      "startTime": "2024-01-14T23:15:00.000",
      "endTime": "2024-01-15T07:20:00.000",
      "minutesAsleep": 445,
      "minutesAwake": 25,
      "minutesToFallAsleep": 12,
      "timeInBed": 485,
      "isMainSleep": true,
      "type": "stages",
      "levels": {
        "data": [
          {"dateTime": "2024-01-14T23:15:00.000", "level": "light", "seconds": 2400},
          {"dateTime": "2024-01-14T23:55:00.000", "level": "deep", "seconds": 3300}
        ],
        "summary": {
          "deep": {"count": 2, "minutes": 95},
          "light": {"count": 5, "minutes": 245},
          "rem": {"count": 3, "minutes": 90},
          "wake": {"count": 3, "minutes": 25},
          "asleep": {"count": 1, "minutes": 430}
        }
      }
    }
  ]
}

```

Both `"type": "stages"` (with REM/deep/light) and `"type": "classic"` (restless/asleep/awake) are supported. The converter correctly handles the summary object where `"asleep"` is the total and `"deep"` , `"light"` , `"rem"` are breakdowns — no double-counting.

Classic sleep example (older devices):

```
{
  "sleep": [
    {
      "dateOfSleep": "2024-06-10",
      "duration": 28200000,
      "efficiency": 87,
      "startTime": "2024-06-09T23:45:00.000",
      "endTime": "2024-06-10T07:30:00.000",
      "minutesAsleep": 430,
      "minutesAwake": 40,
      "minutesToFallAsleep": 18,
      "timeInBed": 470,
      "isMainSleep": true,
      "type": "classic",
      "levels": {
        "data": [
          {"dateTime": "2024-06-09T23:45:00.000", "level": "restless", "seconds": 600},
          {"dateTime": "2024-06-10T00:00:00.000", "level": "asleep", "seconds": 3600},
          {"dateTime": "2024-06-10T01:00:00.000", "level": "restless", "seconds": 300},
          {"dateTime": "2024-06-10T01:05:00.000", "level": "asleep", "seconds": 7200},
          {"dateTime": "2024-06-10T03:05:00.000", "level": "awake", "seconds": 120},
          {"dateTime": "2024-06-10T03:07:00.000", "level": "asleep", "seconds": 10800},
          {"dateTime": "2024-06-10T06:07:00.000", "level": "restless", "seconds": 480},
          {"dateTime": "2024-06-10T06:15:00.000", "level": "awake", "seconds": 900}
        ],
        "shortData": [],
        "summary": {
          "restless": {"count": 3, "minutes": 23},
          "awake": {"count": 2, "minutes": 17},
          "asleep": {"count": 3, "minutes": 430}
        }
      }
    }
  ]
}
```

Activities: Daily Summary (`activities-*.json` in Global Export Data)

```
{
  "activities": [
    {
      "activityName": "Morning Run",
      "activityTypeId": 90009,
      "activeDuration": 2700000,
      "originalDuration": 2750000,
```

```

    "calories": 380,
    "steps": 7200,
    "distance": 8500,
    "distanceUnit": "METERS",
    "startTime": "2024-01-15T07:30:00.000",
    "heartRateZones": [
      {"name": "Out of Range", "min": 30, "max": 90, "minutes": 8},
      {"name": "Fat Burn", "min": 91, "max": 140, "minutes": 22},
      {"name": "Cardio", "min": 141, "max": 170, "minutes": 14},
      {"name": "Peak", "min": 171, "max": 220, "minutes": 1}
    ]
  }
]
}

```

Activities: Detailed Workout (Physical Activity/)

The same format as above but stored in a separate folder by Google Takeout. These often contain more detail including GPS data (though GPS trackpoints are not yet extracted — see limitations).

Steps, Calories, Distance (steps-*.json , calories-*.json , distance-*.json)

```

[
  {"dateTime": "2024-01-15", "value": "12400"}
]

```

These are daily totals. The `value` field is a string that is parsed as a float. Distance units depend on user locale settings (kilometers or miles).

Weight / Body Composition (weight-*.json)

```

[
  {"dateTime": "2024-01-15", "value": {"weight": 75.5, "bmi": 23.5, "fat": 18.0}}
]

```

Timestamps can be date-only ("2024-01-15") or datetime ("2024-01-20T08:00:00"). Both `Body/` and `Global Export Data/` locations are scanned.

Garmin Output Formats

TCX (Training Center XML)

Garmin's native XML format for activities. Supports sport type, duration, distance, calories, lap data, and per-trackpoint heart rate.

```
<?xml version='1.0' encoding='UTF-8'?>
<TrainingCenterDatabase xmlns="http://www.garmin.com/xmlschemas/TrainingCenterDatabase/v2">
  <Activities>
    <Activity Sport="Running">
      <Id>2024-01-15T07:30:00Z</Id>
      <Lap StartTime="2024-01-15T07:30:00Z">
        <TotalTimeSeconds>2750.0</TotalTimeSeconds>
        <DistanceMeters>8500.0</DistanceMeters>
        <Calories>380</Calories>
        <Intensity>Active</Intensity>
        <TriggerMethod>Manual</TriggerMethod>
        <AverageHeartRateBpm><Value>126</Value></AverageHeartRateBpm>
        <MaximumHeartRateBpm><Value>155</Value></MaximumHeartRateBpm>
        <Track>
          <Trackpoint>
            <Time>2024-01-15T07:30:00Z</Time>
            <HeartRateBpm><Value>95</Value></HeartRateBpm>
          </Trackpoint>
        </Track>
      </Lap>
    </Activity>
  </Activities>
</TrainingCenterDatabase>
```

Sport types emitted: Running, Walking, Hiking, Treadmill, Biking, Stationary Biking, Swimming, Elliptical, Strength, Yoga, Golf, Tennis, CrossFit, HIIT, Other, and more. Maps directly from Fitbit's `activityTypeId` .

CSV: Body Composition

Formatted for Garmin Connect's body composition import tool:

Date	Weight	BMI	BodyFat	BoneMass	MuscleMass	BodyWater
2024-01-15	75.50	23.5	18.0			
2024-01-16	75.80	23.6	18.2			

Garmin Connect maps these columns as:

- **Date** → date of measurement
- **Weight** → body weight in kg
- **BMI** → body mass index (auto-calculated by Garmin if empty)
- **BodyFat** → body fat percentage
- **BoneMass, MuscleMass, BodyWater** → left empty (not available from Fitbit export)

CSV: Sleep

Reference CSV for your records. Garmin Connect does not support sleep data import:

Date,Start Time,End Time,Duration (min),Minutes Asleep,Minutes Awake,Efficiency,Time in Bed (min),Minutes
2024-01-15,2024-01-14 23:15:00,2024-01-15 07:20:00,490,430,25,93,485,12

CSV: Daily Summary

Daily totals for steps, calories, and distance:

Date,Steps,Calories,Distance (m)
2024-01-15,12400,2520.0,14.3

Note: Distance units in Fitbit's `distance-*.json` files are locale-dependent (km for metric, miles for imperial). The converter passes the raw value through. Adjust units in the CSV if needed.

Feature Comparison: Fitbit vs Garmin

Feature	Fitbit	Garmin	Import
Running/Walking/Cycling	activityName + typeId + zones	Sport type + HR zones	□ TCX
Heart rate (intraday)	<code>{bpm, confidence}</code> per sample	Per-trackpoint in TCX/FIT	□ TCX
Heart rate (daily)	Resting HR + zone minutes	Resting HR + intensity mins	△ TCX lap only
Sleep stages	REM / Deep / Light / Awake	REM / Deep / Light / Awake	□
Sleep score	Efficiency %	Sleep score (1-100)	□
Weight & BMI	Manual or scale logs	Manual or scale logs	□ CSV
Body fat %	With Aria scale	With Index scale	□ CSV
Steps	Daily total	Daily total	□
Calories	Active + BMR combined	Active + Resting split	□
Distance	Daily total	Daily total (GPS-based)	□
GPS tracks	Per workout	Per activity .FIT/.GPX	△ partial
VO2 Max	Cardio Fitness Score	VO2 Max estimate	□
Floors climbed	Barometric	Barometric (most devices)	□

Feature	Fitbit	Garmin	Import
Food / Nutrition	Barcode scanning	Manual (Connect)	☐
Water intake	Manual log	Manual log	☐
Menstrual health	Cycle tracking	Cycle tracking	☐
SpO2 / Blood oxygen	Nightly average	Pulse Ox	☐
Stress / HRV	Stress score	Stress + Body Battery	☐

Conversion Mapping

Fitbit Activity Type → Garmin Sport

Fitbit activityTypeId	Fitbit activityName (example)	Garmin Sport
90009	Run, Morning Run	Running
90010	Walk, Evening Walk	Walking
90011	Hike	Hiking
90012	Treadmill	Running (indoor)
90013	Cycling, Bike	Biking
90014	Stationary Bike, Spinning	Stationary Biking
90015	Swim, Swimming	Swimming
90016	Elliptical	Elliptical
90017	Weights, Strength	Strength
90018	Yoga	Yoga
90019	Tennis	Tennis
90020	Basketball	Basketball
90022	Golf	Golfing
90027	Meditation	Other
90028	Cardio	Other
90033	HIIT	Other
unknown	Run	Running

Fitbit activityTypeId	Fitbit activityName (example)	Garmin Sport
<i>unknown</i>	Walk	Walking
<i>unknown</i>	Swim	Swimming

When `activityTypeId` is not present, the converter falls back to keyword matching on the activity name.

Activity Duration

- Fitbit stores durations in **milliseconds** (both `activeDuration` and `originalDuration`)
- The converter uses the larger of the two, divided by 1000 to get **seconds**
- Garmin TCX stores duration as `<TotalTimeSeconds>` (decimal seconds)

Heart Rate

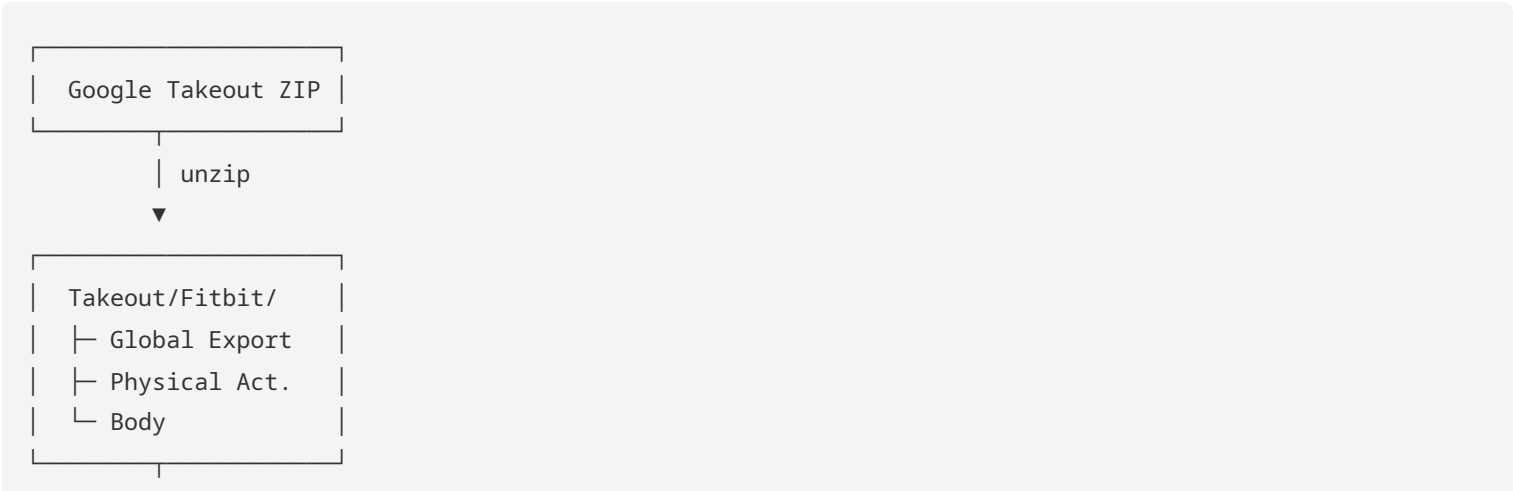
- **Per-trackpoint:** Intraday HR samples from `heart_rate-*.json` files that overlap with the activity time window are matched and added as `<Trackpoint><HeartRateBpm>` elements in the TCX
- **Average HR:** Calculated from matched trackpoint BPM values, or taken from the activity's `heartRate` object if present
- **Max HR:** Derived from matched samples (actual peak), not zone boundaries
- **Matching window:** 30 seconds before activity start to 30 seconds after activity end

Distance & Calories

- **Distance:** Passed through in meters. Both the `activities-*.json` (with `distanceUnit: "METERS"`) and `distance-*.json` are read
- **Calories:** Passed through directly as integer from the activity record

How the Integration Works

Architecture



| python -m fitbit2garmin



fitbit2garmin (reader + converter)

1. Scan directory for *.json files
grouped by type (heart_rate,
sleep, activities, steps, etc.)

2. Parse each JSON file
Handle date formats, field names,
and structural variations across
different Takeout versions

3. Cross-reference data
Match HR samples to activities by
time range overlap

4. Generate output files
TCX for activities (with embedded
HR trackpoints)
CSV for body, sleep, daily summary



garmin_output/

├─ activities/ ← TCX files (import to Garmin Connect)

│ └─ 20240115_*

│ └─ 20240116_*

│ └─ ...

├─ body_composition.csv ← Import: Health Stats → Body Comp

├─ sleep.csv ← Reference only

└─ daily_summary.csv ← Reference only

Data Flow

- Reader** (`reader.py`): Walks the input directory tree, identifies JSON files by filename prefix (`heart_rate-` , `sleep-` , `activities-` , `steps-` , `calories-` , `distance-` , `weight-` , `body-`), and parses each into the internal data models (`Activity` , `HeartRateSample` , `SleepSession` , `WeightLog` , `DailySummary`).
- Converter** (`converter.py`): Takes the raw parsed data and cross-references it. The key transformation is matching global HR samples to activities. It also optionally builds synthetic "Other" activities for days that have HR data but no recorded workout (the `--hr-only` flag).
- Writer** (`writer.py`): Serializes the processed data into Garmin-compatible formats using XML (`xml.etree.ElementTree`) for TCX and the `csv` module for CSV files.

HR Matching Algorithm

For each activity:

1. Calculate time window: [start - 30s, end + 30s]
2. Find all HeartRateSamples within that window
3. If samples found:
 - Attach them as Trackpoints (in chronological order)
 - Compute average HR from sample BPM values
 - Compute max HR as the peak BPM value
 - Override any zone-based estimates with actual data

For each day with HR samples but no activity (--hr-only):

1. Collect all samples for that date
2. If ≥ 10 samples exist and span ≥ 60 seconds:
 - Create a synthetic "Other" activity
 - Duration = span of samples
 - All samples become trackpoints
 - avg/max HR from sample data

Installation

```
# No installation required – run directly:
python -m fitbit2garmin /path/to/Takeout/Fitbit -o ./output

# Or install with pip for use as a command:
pip install -e /path/to/fitbit2garmin
fitbit2garmin /path/to/Takeout/Fitbit -o ./output
```

Requires Python 3.10+. No external dependencies — uses only the standard library.

Usage

```
usage: python -m fitbit2garmin [-h] [-o OUTPUT_DIR] [--no-activities]
                                [--hr-only] [--no-weight] [--no-sleep]
                                [--no-summary] [-v] [--version]
                                input_dir
```

Convert Fitbit data export to Garmin-compatible formats

positional arguments:

input_dir	Path to Fitbit export directory (extracted Takeout/Fitbit or similar)
-----------	---

options:

-h, --help	show this help message and exit
-o, --output-dir OUTPUT_DIR	Output directory (default: ./garmin_output)
--no-activities	Skip activity-to-TCX conversion

--hr-only	Create TCX files for days with HR data even without a recorded activity
--no-weight	Skip body composition CSV export
--no-sleep	Skip sleep CSV export
--no-summary	Skip daily summary CSV export
-v, --verbose	Verbose output
--version	show program's version number and exit

Examples

```
# Basic conversion (all categories)
python -m fitbit2garmin ~/Downloads/Takeout/Fitbit -o ./garmin_data

# Activities + weight only, verbose logging
python -m fitbit2garmin ~/Downloads/Takeout/Fitbit -o ./garmin_data \
    --no-sleep --no-summary -v

# Generate TCX for all days with heart rate data (even non-activity days)
python -m fitbit2garmin ~/Downloads/Takeout/Fitbit -o ./garmin_data --hr-only

# TCX files only, no CSVs
python -m fitbit2garmin ~/Downloads/Takeout/Fitbit -o ./garmin_data \
    --no-weight --no-sleep --no-summary
```

Importing into Garmin Connect

Activities (TCX files)

1. Go to [Garmin Connect](#) and sign in
2. Click **Activities** in the top navigation bar
3. Click **Import Data** (top-right, gear icon dropdown)
4. Select **Import from file...**
5. Choose one or more `.tcx` files from `output/activities/`
6. Click **Upload**

Repeat for each activity (Garmin does not support batch upload). Each uploaded activity will appear in your history with duration, distance, calories, heart rate data, and sport type.

Body Composition (CSV)

1. In Garmin Connect, go to **Health Stats** (in the main menu)
2. Go to **Body Composition**
3. Click the gear icon → **Import Data**
4. Select the `body_composition.csv` file

5. Map columns:

- **Date** → Date
- **Weight** → Weight (kg)
- **Body Fat** → Body Fat %
- **BMI** → BMI

6. Click **Import**

Garmin will show these measurements alongside your Garmin-device readings.

Sleep and Daily Summary (CSV)

Garmin Connect **does not** support importing sleep or daily step/calorie/distance data via CSV. These files are provided for your personal records and can be opened in any spreadsheet application (Excel, Google Sheets, Numbers).

File Format Specifications

TCX Schema (simplified)

```
TrainingCenterDatabase
├── Activities
│   ├── Activity (@Sport)
│   │   ├── Id (ISO 8601 timestamp)
│   │   ├── Lap (@StartTime)
│   │   │   ├── TotalTimeSeconds (decimal seconds)
│   │   │   ├── DistanceMeters (decimal meters)
│   │   │   ├── Calories (integer kcal)
│   │   │   ├── Intensity ("Active" | "Resting")
│   │   │   ├── TriggerMethod ("Manual" | "Auto")
│   │   │   ├── AverageHeartRateBpm
│   │   │   │   ├── Value
│   │   │   ├── MaximumHeartRateBpm
│   │   │   │   ├── Value
│   │   │   ├── Track
│   │   │   │   ├── Trackpoint (repeating)
│   │   │   │   │   ├── Time (ISO 8601)
│   │   │   │   │   ├── HeartRateBpm
│   │   │   │   │   │   ├── Value
```

Filename convention: `YYYYMMDD_HHMMSS_Sport.tcx`

Body Composition CSV

Date,Weight,BMI,BodyFat,BoneMass,MuscleMass,BodyWater

- Date: YYYY-MM-DD
- Weight: kg (decimal)
- BMI: decimal
- BodyFat: percentage (decimal)
- BoneMass / MuscleMass / BodyWater: left empty (data not available from Fitbit)

Limitations

Limitation	Details
No GPS trackpoints	Fitbit's <code>activities-*.json</code> exports do not include GPS coordinates or route data. TCX files contain time + heart rate only, not lat/lon
No sleep import	Garmin Connect does not expose a sleep data import API. Sleep CSVs are for personal reference only
No batch upload to Garmin	Garmin Connect's web UI requires selecting TCX files one-by-one
Distance units ambiguous	Daily summary distance from <code>distance-*.json</code> has no unit field. Value is passed as-is (km for metric users, miles for imperial)
No SpO2 / HRV / Stress	These sensor data types from Fitbit are not covered by this converter
No food / nutrition	Fitbit food logs are not converted
No floors	Fitbit floor data (<code>floors-*.json</code>) is not currently processed
Heart rate confidence	Fitbit's <code>confidence</code> field (0-3) is parsed but not used in TCX output

FAQ

Does this work with Google Health (the renamed Fitbit app)?

Yes. Google Takeout still labels the data category as "Fitbit" even after the rebrand. The folder structure and JSON formats are unchanged.

Can I import TCX files on mobile?

Garmin Connect Mobile does not support TCX import. Use the web version at connect.garmin.com on a desktop browser.

My activity durations look wrong — what's activeDuration vs originalDuration?

Fitbit records two durations:

- `activeDuration` : time spent actually moving (excluding pauses, rests)
- `originalDuration` : total elapsed time including pauses

The converter uses the **larger** of the two values, which typically gives the total time (most appropriate for Garmin's activity representation).

Can I run this on my Pixel Watch data?

Yes. Pixel Watch data is stored in the same Fitbit/Google Health infrastructure and exports via Google Takeout with the same JSON format.

What about the July 2026 data deletion?

If you never migrated your legacy Fitbit account to a Google account, Google will start permanently deleting un-migrated data on **15 July 2026**. Export your data via Takeout before then. Already-migrated accounts are not affected.

Why don't I see HR data in my imported activities?

The converter matches heart rate samples to activities by time overlap. If your HR files use a different date format than expected, or if the timestamps don't align with the activity time window, HR may not be embedded. Run with `-v` to see how many HR samples were found and matched.

garmin2fitbit — Converter Documentation

garmin2fitbit

Convert Garmin data exports into Fitbit-compatible JSON format for archival or API-based migration.

Fitbit has **no official bulk import feature**. Unlike Garmin Connect (which accepts TCX and CSV), the only way to get historical data into Fitbit is through the **Fitbit Web API** or **third-party sync apps** (RunGap, HealthSync) that use it. This converter prepares your Garmin data in the right format.

Table of Contents

- [Feature & Import Feasibility](#)
- [Garmin Input Formats](#)
- [Fitbit Output Formats](#)
- [Installation](#)
- [Usage](#)
- [Third-Party Import Options](#)
- [Setting Up Garmin Export](#)
- [Limitations](#)

Feature & Import Feasibility

Garmin Data	Fitbit Output	Import into Fitbit	Path	--- --- --- --- ---	Activities (TCX: running, walking, cycling, etc.)	<code>activities.json</code> + TCX passthrough	△ Possible	RunGap, HealthSync, or direct Fitbit API POST /1/user/-/activities.json	Heart rate (per-trackpoint from TCX)	Embedded in <code>activities.json</code> + zones	☐	No API endpoint to upload HR data; only third-party tools that accept TCX can carry it	Weight & BMI (CSV)	<code>weight.json</code>	△ Possible	Fitbit API POST /1/user/-/body/log/weight.json	Body fat % (CSV)	<code>weight.json</code> (with <code>fat</code> field)	△ Possible	Fitbit API POST /1/user/-/body/log/fat.json	Sleep (CSV)	<code>sleep.json</code>	△ Possible	Fitbit API POST /1.2/user/-/sleep.json — classic sleep only (no REM/deep/light stages)	Steps (daily CSV)	<code>activities-steps.json</code>	☐	No import path (Fitbit doesn't expose step logging in the API)	Calories (daily CSV)	<code>activities-calories.json</code>	☐	No import path	Distance (daily CSV)	<code>activities-distance.json</code>	☐	No import path	Floors (daily CSV)	Not converted	☐	No import path	Cadence (from TCX)	Steps derived as <code>cadence × duration</code>	☐	Not importable; included in
-------------	---------------	--------------------	------	---------------------	--	--	----------------------------	---	---	--	--------------------------	--	-------------------------------	--------------------------	----------------------------	--	-------------------------	--	----------------------------	---	--------------------	-------------------------	----------------------------	--	--------------------------	------------------------------------	--------------------------	--	-----------------------------	---------------------------------------	--------------------------	----------------	-----------------------------	---------------------------------------	--------------------------	----------------	---------------------------	---------------	--------------------------	----------------	---------------------------	--	--------------------------	-----------------------------

Garmin Input Formats

The converter reads TCX files and CSV exports from Garmin Connect.

TCX Activities (`activities/*.tcx`)

Standard Garmin TCX v2 format with per-trackpoint heart rate:

```
<?xml version='1.0' encoding='UTF-8'?>
<TrainingCenterDatabase xmlns="http://www.garmin.com/xmlschemas/TrainingCenterDatabase/v2">
  <Activities>
    <Activity Sport="Running">
      <Id>2024-01-15T07:30:00Z</Id>
      <Lap StartTime="2024-01-15T07:30:00Z">
        <TotalTimeSeconds>2750.0</TotalTimeSeconds>
        <DistanceMeters>8500.0</DistanceMeters>
        <Calories>380</Calories>
        <Intensity>Active</Intensity>
        <TriggerMethod>Manual</TriggerMethod>
        <AverageHeartRateBpm><Value>126</Value></AverageHeartRateBpm>
        <MaximumHeartRateBpm><Value>155</Value></MaximumHeartRateBpm>
        <Track>
          <Trackpoint>
            <Time>2024-01-15T07:30:00Z</Time>
            <HeartRateBpm><Value>95</Value></HeartRateBpm>
          </Trackpoint>
        </Track>
      </Lap>
    </Activity>
  </Activities>
</TrainingCenterDatabase>
```

The reader also extracts `<Cadence>` from trackpoints and derives a step count as `cadence × duration (min)` .

Body Composition CSV

Garmin Connect data export format:

```
Date,Weight,BMI,BodyFat,BoneMass,MuscleMass,BodyWater
2024-01-15,75.5,23.9,15.8,0,0,0
```

Sleep CSV

Expected columns: `Date,Sleep Start,Sleep End,Duration (min),Minutes Asleep,Minutes Awake`

```
Date,Sleep Start,Sleep End,Duration (min),Minutes Asleep,Minutes Awake,Efficiency
2024-01-15,2024-01-14 22:45:00,2024-01-15 06:30:00,465,435,30,93
```

Daily Summary CSV

Expected columns: `Date,Steps,Calories,Distance,Floors,Minutes Sedentary,Minutes Lightly Active,...`

```
Date,Steps,Calories,Distance,Floors,Minutes Sedentary,Minutes Lightly Active
2024-01-15,12400,2520,14300,10,360,240
```

Fitbit Output Formats

Activities JSON (`activities.json`)

Matches the Google Takeout activity structure:

```
{
  "activities": [
    {
      "activityName": "Running 2024-01-15 07:30",
      "activityTypeId": 90009,
      "activeDuration": 2750000,
      "calories": 380,
      "distance": 8500.0,
      "distanceUnit": "METERS",
      "steps": 0,
      "startTime": "2024-01-15T07:30:00.000",
      "duration": 2750000,
      "heartRateZones": [
        {"name": "Fat Burn", "min": 95, "max": 134, "minutes": 4, "caloriesOut": 0},
        {"name": "Cardio", "min": 135, "max": 159, "minutes": 3, "caloriesOut": 0}
      ],
      "heartRate": {"average": 126, "max": 155}
    }
  ]
}
```

Sport type mapping:

Garmin Sport	Fitbit activityTypeId
Running	90009
Walking	90010

Garmin Sport	Fitbit activityTypeId
Hiking	90011
Biking / Cycling	90013
Swimming	90015
Elliptical	90016
Weights / Strength	90017
Yoga	90018
Cardio / Other	90028

Weight JSON (`weight.json`)

```
[
  {
    "dateTime": "2024-01-15",
    "value": {
      "weight": 75.5,
      "bmi": 23.9,
      "fat": 15.8
    }
  }
]
```

Sleep JSON (`sleep.json`)

```
{
  "sleep": [
    {
      "dateOfSleep": "2024-01-15",
      "duration": 27900000,
      "startTime": "2024-01-14T22:45:00.000",
      "endTime": "2024-01-15T06:30:00.000",
      "minutesAsleep": 435,
      "minutesAwake": 30,
      "efficiency": 0,
      "isMainSleep": true,
      "timeInBed": 465,
      "minutesToFallAsleep": 0,
      "levels": {
        "summary": {},
        "data": []
      }
    }
  ]
}
```

Daily Field JSONs (`activities-{steps,calories,distance}.json`)

```
[
  {
    "dateTime": "2024-01-15",
    "value": "12400"
  }
]
```

These match the format that `steps-*.json` , `calories-*.json` , and `distance-*.json` use in a real Fitbit Takeout export.

TCX Passthrough (`activities/*.tcx` , with `--tcx` flag)

Identical to the original Garmin TCX files, regenerated for direct use with third-party upload tools.

Installation

```
# No installation required – run directly:
python -m garmin2fitbit /path/to/garmin/export -o ./fitbit_output

# Or install with pip for use as a command:
pip install -e /path/to/fitbit2garmin
garmin2fitbit /path/to/garmin/export -o ./fitbit_output
```

Requires Python 3.10+. No external dependencies.

Usage

usage: python -m garmin2fitbit [-h] [-o OUTPUT_DIR] [--no-activities]
 [--tcx] [--no-weight] [--no-sleep]
 [--no-summary] [-v] [--version]
 input_dir

Convert Garmin data export to Fitbit-compatible formats

positional arguments:

input_dir Directory with Garmin TCX files (in activities/)
 and CSV exports

options:

-h, --help show this help message and exit

-o, --output-dir OUTPUT_DIR

Output directory (default: ./fitbit_output)

--no-activities Skip Fitbit JSON activities output

--tcx Also generate TCX passthrough files (for Fitbit
 API upload)

--no-weight Skip body composition JSON export

--no-sleep Skip sleep JSON export

--no-summary Skip daily summary JSON export

-v, --verbose Verbose output

--version show program's version number and exit

Examples

```
# Basic conversion – all data to Fitbit JSON
python -m garmin2fitbit ~/Downloads/Garmin/Export -o ./fitbit_data

# Also generate TCX passthrough files (for third-party upload tools)
python -m garmin2fitbit ~/Downloads/Garmin/Export -o ./fitbit_data --tcx

# Activities + weight only
python -m garmin2fitbit ~/Downloads/Garmin/Export -o ./fitbit_data \
    --no-sleep --no-summary

# TCX passthrough only (no Fitbit JSON at all)
python -m garmin2fitbit ~/Downloads/Garmin/Export -o ./fitbit_data \
    --no-weight --no-sleep --no-summary --tcx

# Round-trip: Fitbit → Garmin TCX → Fitbit JSON
python -m fitbit2garmin ~/Takeout/Fitbit -o ./garmin_output
python -m garmin2fitbit ./garmin_output -o ./back_to_fitbit --tcx
```

Third-Party Import Options

Since Fitbit has no native bulk import, here are the practical options:

Tool	Platform	Cost	Supports (Write to Fitbit)	--- --- --- --- ---	Health Sync	Android, iOS	
Paid (lifetime license available)	Steps, heart rate (daily), sleep, weight, body fat, activities; via Google Health APIs	myFitnessSync	iOS	Paid subscription	Steps, weight, body fat, sleep, water (5 fields); Apple Health → Fitbit	RunGap	iOS
Subscription after trial	Activities (workouts), daily summaries; 50+ platforms bridged	Fitbit Web API	Any (scripts)	Free	Activities, weight, body fat, sleep (classic only); see API section below	Manual entry	Fitbit mobile app
Free	Everything (one-by-one, tedious for bulk)						

Getting a Fitbit API Token

Fitbit uses OAuth 2.0. You need a **personal app** to get a token:

1. Go to dev.fitbit.com → **Register an App**
2. Set **OAuth 2.0 Application Type** to **Personal**
3. Set **Redirect URL** to `https://localhost` (or anything valid)
4. After registering, note your **Client ID** and **Client Secret**

Then get a token with curl (requires a browser step):

```
# 1. Open this URL in your browser, authorize, then copy the code from the redirect
open "https://www.fitbit.com/oauth2/authorize?response_type=code&client_id=YOUR_CLIENT_ID&redirect_uri=htt

# 2. Exchange the code for an access token
curl -X POST https://api.fitbit.com/oauth2/token \
```

```
-H "Authorization: Basic $(echo -n 'YOUR_CLIENT_ID:YOUR_CLIENT_SECRET' | base64)" \
-d "grant_type=authorization_code" \
-d "code=CODE_FROM_STEP_1" \
-d "redirect_uri=https://localhost"
```

```
# 3. Use the returned access_token in the examples below
TOKEN="your_access_token_here"
```

Personal apps get tokens valid for 1 year. Save your `refresh_token` to renew it.

Upload Activities (curl)

The `activities.json` output from `garmin2fitbit` is an array. Upload each entry via the Fitbit API:

```
TOKEN="your_access_token_here"

# Upload one activity at a time
curl -X POST "https://api.fitbit.com/1/user/-/activities.json" \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "activityId": 90009,
  "startTime": "2024-01-15T07:30:00",
  "durationMillis": 2750000,
  "distance": 8500,
  "distanceUnit": "METERS",
  "manualCalories": 380
}'
```

Batch script to upload all activities from `activities.json` :

```
TOKEN="your_access_token_here"

cat fitbit_output/activities.json | jq -c '.activities[]' | while read -r act; do
  activityId=$(echo "$act" | jq '.activityTypeId')
  startTime=$(echo "$act" | jq -r '.startTime')
  durationMillis=$(echo "$act" | jq '.duration')
  distance=$(echo "$act" | jq '.distance')
  calories=$(echo "$act" | jq '.calories')
  name=$(echo "$act" | jq -r '.activityName')

  echo "Uploading: $name"

  curl -s -X POST "https://api.fitbit.com/1/user/-/activities.json" \
  -H "Authorization: Bearer $TOKEN" \
  -d "activityId=$activityId" \
  -d "startTime=$startTime" \
  -d "durationMillis=$durationMillis" \
  -d "distance=$distance" \
  -d "distanceUnit=METERS" \
  -d "manualCalories=$calories"
```

```
# Rate limit: 150 requests per hour
sleep 5
done
```

Upload Body Composition (curl)

```
TOKEN="your_access_token_here"

# Upload weight log
curl -X POST "https://api.fitbit.com/1/user/-/body/log/weight.json" \
  -H "Authorization: Bearer $TOKEN" \
  -d "weight=75.5" \
  -d "date=2024-01-15" \
  -d "time=08:00"

# Upload body fat percentage
curl -X POST "https://api.fitbit.com/1/user/-/body/log/fat.json" \
  -H "Authorization: Bearer $TOKEN" \
  -d "fat=15.8" \
  -d "date=2024-01-15" \
  -d "time=08:00"
```

Batch script from `weight.json` :

```
TOKEN="your_access_token_here"

cat fitbit_output/weight.json | jq -c '.[[]]' | while read -r entry; do
  date=$(echo "$entry" | jq -r '.dateTime')
  weight=$(echo "$entry" | jq -r '.value.weight')
  fat=$(echo "$entry" | jq -r '.value.fat // empty')

  echo "Uploading weight for $date: ${weight}kg"

  curl -s -X POST "https://api.fitbit.com/1/user/-/body/log/weight.json" \
    -H "Authorization: Bearer $TOKEN" \
    -d "weight=$weight" \
    -d "date=$date" \
    -d "time=08:00"

  if [ -n "$fat" ]; then
    curl -s -X POST "https://api.fitbit.com/1/user/-/body/log/fat.json" \
      -H "Authorization: Bearer $TOKEN" \
      -d "fat=$fat" \
      -d "date=$date" \
      -d "time=08:00"
  fi

  sleep 3
done
```

Upload Sleep (curl)

The Fitbit API can create sleep logs via `POST /1.2/user/-/sleep.json` . Note that this creates **classic sleep** (start time + duration) — no REM/deep/light stage data is preserved.

```
TOKEN="your_access_token_here"

# Upload a single sleep log
curl -X POST "https://api.fitbit.com/1.2/user/-/sleep.json" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "date": "2024-01-15",
    "startTime": "2024-01-14T22:45:00",
    "duration": 27900000
  }'
```

Batch script from `sleep.json` :

```
TOKEN="your_access_token_here"

cat fitbit_output/sleep.json | jq -c '.sleep[]' | while read -r entry; do
  date=$(echo "$entry" | jq -r '.dateOfSleep')
  startTime=$(echo "$entry" | jq -r '.startTime' | sed 's/\.000$//')
  duration=$(echo "$entry" | jq -r '.duration')

  echo "Uploading sleep for $date"

  curl -s -X POST "https://api.fitbit.com/1.2/user/-/sleep.json" \
    -H "Authorization: Bearer $TOKEN" \
    -H "Content-Type: application/json" \
    -d "{\"date\":\"$date\",\"startTime\":\"$startTime\",\"duration\":$duration}"

  sleep 5
done
```

Full Python Upload Script

Save as `upload_to_fitbit.py` alongside your `fitbit_output/` directory:

```
#!/usr/bin/env python3
"""Upload garmin2fitbit output to Fitbit via the Web API."""

import json
import os
import time
import requests

TOKEN = "your_access_token_here"
HEADERS = {"Authorization": f"Bearer {TOKEN}"}
```

```
OUTPUT_DIR = "fitbit_output"
```

```
def upload_activities():
    path = os.path.join(OUTPUT_DIR, "activities.json")
    if not os.path.exists(path):
        return
    with open(path) as f:
        data = json.load(f)
    for act in data.get("activities", []):
        payload = {
            "activityId": act["activityTypeId"],
            "startTime": act["startTime"],
            "durationMillis": act["duration"],
            "distance": act["distance"],
            "distanceUnit": "METERS",
            "manualCalories": act["calories"],
        }
        print(f" → {act['activityName']} ...", end=" ")
        r = requests.post(
            "https://api.fitbit.com/1/user/-/activities.json",
            headers=HEADERS, data=payload,
        )
        print("OK" if r.ok else f"FAIL ({r.status_code})")
        time.sleep(5)

def upload_weight():
    path = os.path.join(OUTPUT_DIR, "weight.json")
    if not os.path.exists(path):
        return
    with open(path) as f:
        entries = json.load(f)
    for entry in entries:
        date = entry["dateTime"]
        val = entry["value"]
        print(f" → Weight {date} ({val['weight']} kg) ...", end=" ")
        r = requests.post(
            "https://api.fitbit.com/1/user/-/body/log/weight.json",
            headers=HEADERS,
            data={"weight": val["weight"], "date": date, "time": "08:00"},
        )
        print("OK" if r.ok else f"FAIL ({r.status_code})")

    if val.get("fat"):
        print(f" → Body fat {date} ({val['fat']}%) ...", end=" ")
        r = requests.post(
            "https://api.fitbit.com/1/user/-/body/log/fat.json",
            headers=HEADERS,
            data={"fat": val["fat"], "date": date, "time": "08:00"},
        )
        print("OK" if r.ok else f"FAIL ({r.status_code})")
```

```
time.sleep(3)
```

```
def upload_sleep():
    path = os.path.join(OUTPUT_DIR, "sleep.json")
    if not os.path.exists(path):
        return
    with open(path) as f:
        data = json.load(f)
    for entry in data.get("sleep", []):
        payload = {
            "date": entry["dateOfSleep"],
            "startTime": entry["startTime"].rstrip(".000"),
            "duration": entry["duration"],
        }
        print(f" → Sleep {payload['date']} ...", end=" ")
        r = requests.post(
            "https://api.fitbit.com/1.2/user/-/sleep.json",
            headers=HEADERS, json=payload,
        )
        print("OK" if r.ok else f"FAIL ({r.status_code})")
        time.sleep(5)

if __name__ == "__main__":
    print("Uploading activities...")
    upload_activities()
    print("Uploading body composition...")
    upload_weight()
    print("Uploading sleep...")
    upload_sleep()
    print("Done.")
```

Rate Limits

Limit	Value
Requests per hour (per user)	150
Requests per day (per user)	30,000

The batch scripts above include `sleep` calls to stay within limits. For large exports, add longer delays or split across multiple days.

Data Type Import Feasibility by Tool

Data	Health Sync	myFitnessSync	RunGap	Fitbit API	Manual
Steps (daily)	☐	☐	☐	☐	☐
Heart rate (daily/resting)	☐	☐	☐	☐	☐

Data	Health Sync	myFitnessSync	RunGap	Fitbit API	Manual
Heart rate (intraday)	☐	☐	☐	☐	☐
Sleep (classic)	☐	☐	☐	☐	☐
Sleep (stages)	☐	☐	☐	☐	☐
Menstrual cycle	☐	☐	☐	☐	☐ (app only)
Activities (workouts)	☐	☐	☐	☐	☐
Weight / Body fat	☐	☐	☐	☐	☐

Note: Health Sync uses the newer Google Health APIs (post-Fitbit migration). If your account hasn't migrated, features may differ. myFitnessSync writes via Apple Health → Fitbit bridge and is iOS-only.

Setting Up Garmin Export

1. Go to [Garmin Connect](#) → **Settings** (gear icon) → **Data Management**
2. Click **Export Your Data** and wait for the email
3. Download and extract the ZIP
4. The converter expects:
 - TCX files inside an `activities/` subdirectory under `input_dir`
 - CSV files (body composition, sleep, daily summary) at the top level of `input_dir`

Alternatively, point it at any directory containing TCX files in an `activities/` folder (like the output from `fitbit2garmin`):

```
python -m fitbit2garmin ~/Takeout/Fitbit -o ./garmin_output
python -m garmin2fitbit ./garmin_output -o ./fitbit_formatted --tcx
```

Limitations

Limitation	Details
No bulk import into Fitbit	Fitbit does not provide a CSV/TCX upload feature. The JSON output is for archival or API consumption
Sleep: classic only	The API accepts sleep logs via <code>POST /1.2/user/-/sleep.json</code> , but only classic sleep (start time + duration) — no REM/deep/light stage data

Limitation	Details
Daily steps/calories cannot be imported	Fitbit's API does not support logging daily totals for steps, calories, or distance
Heart rate cannot be imported standalone	No API endpoint exists to upload HR data; it can only be carried by TCX via third-party tools
Menstrual health cannot be imported	Fitbit Web API has no write endpoint for cycle or menstrual health data
No HRV / Stress / SpO2	Garmin's advanced metrics are not converted to Fitbit equivalents
No GPS trackpoints	TCX-to-JSON conversion keeps heart rate data but GPS coordinates are not extracted
Sleep efficiency defaulted to 0	Garmin sleep CSVs don't include an efficiency score; the field is written as 0
